UNIVERSIDAD TECNOLÓGICA DE EL SALVADOR

FACULTAD DE INFORMÁTICA Y CIENCIAS APLICADAS



Asignatura:

Lenguaje Unificado Modelado (UML)

Sección:

01

Practica:

Modelado Dinámico con Diagramas de Máquinas de Estado UML en StarUML.

Integrantes y roles:

Nombre	Rol	Carnet
Víctor Daniel Escobar Morán	Líder de Arquitectura y Modelado UML	25-1473-2025
Jose Francisco Martinez Ramos	Especialista en Reglas de Negocio y Casos de Borde	27-5393-2018
Axel Rodrigo Rodas Serrano	Ingeniero de Documentación Técnica y Matrices QA	25-2374-2025
Oscar Josue Pleitez Lopez	Especialista en Integración, BD y Patrón State	27-0649-2026
Julia Damaris Ruiz Chulo	Defensor de Negocio y Portavoz Técnico (Product Advocate)	17-2493-2026

Entrega:

San Salvador, 22 de octubre

Índice

Introducción 3

Alcance de LogiFlow 360° 3

Fichas Técnicas de las 10 Entidades: 4

Nombre de cada Entidad, Objeto Representado y Diagrama de estados. 4

Matriz de Transición de Estados y Análisis de Casos de Borde y Resiliencia: 9

06. TicketDeSoporte 15

07. SolicitudDeDevolucion 16

08. LoteDeInventario 17

09. CuentaDeConductor 19

10. DisputaBancaria 20

Diseño de Persistencia y Auditoría de Estados 22

Objetivo 22

Propuesta de Implementación de Código (Patrón State): 24

Diagrama de Clase - OrdenDeCompra 24

Lecciones Aprendidas y Conclusiones del Análisis 29

Lecciones aprendidas: 29

Conclusiones del análisis: 29

Introducción

La estabilidad en sistemas empresariales de alta concurrencia depende de un control estricto en las transiciones de estado para evitar fallos transaccionales o pérdidas financieras. Este informe técnico documenta el diseño de las máquinas de estados (SMD) de las 10 entidades críticas de la plataforma **LogiFlow 360°** utilizando UML 2.5 en StarUML. Se integran especificaciones formales con sintaxis de transiciones, eventos temporales after(), diseño de persistencia para auditoría y la implementación en C# mediante el patrón de diseño State, garantizando un sistema robusto, determinista y resiliente.

Alcance de LogiFlow 360°

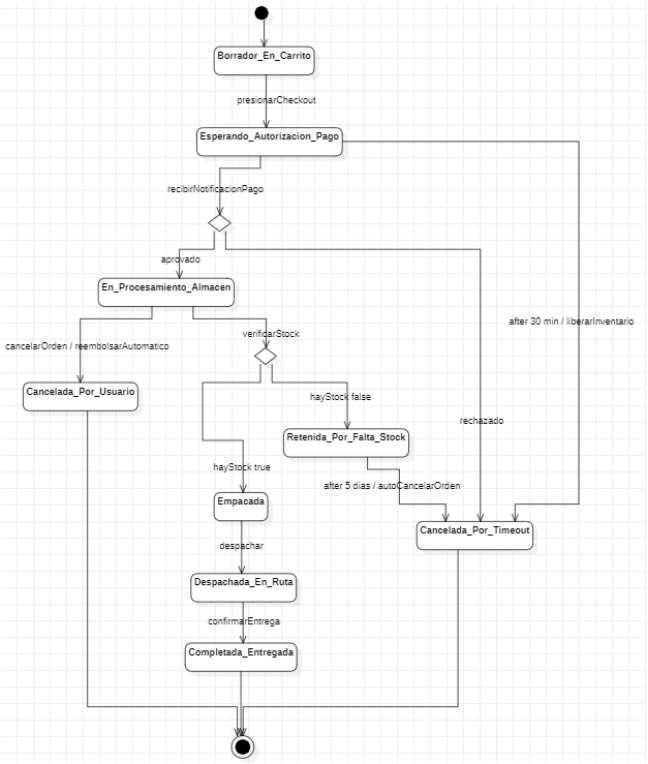
LogiFlow 360° es una plataforma global de comercio electrónico, logística de última milla, casilleros inteligentes IoT y gestión de flotas vehiculares que procesa más de 500,000 operaciones diarias en tiempo real. Su alcance técnico comprende el modelado formal de las diez (10) entidades obligatorias del sistema:

- **OrdenDeCompra**: Ciclo comercial y transaccional de la orden.
- **TransaccionDePago**: Pasarela bancaria, fraudes, 3DS y liquidación.
- **PaqueteFisico**: Cadena de custodia logística desde el hub a destino.
- **VehiculoDeReparto**: Telemetría, mantenimientos y turnos de la unidad.
- **CasilleroInteligente**: Compartimento físico, códigos OTP y hardware.
- **TicketDeSoporte**: SLAs de atención al cliente y escalamientos N1/N2.
- **SolicitudDeDevolucion**: Logística inversa, inspección y reembolso.
- **LoteDeInventario**: Cuarentena, stock activo, mermas y bloqueo.
- **CuentaDeConductor**: Onboarding, antecedentes, suspensión y baja.
- **DisputaBancaria**: Reclamos por contracargo, evidencias y arbitraje.

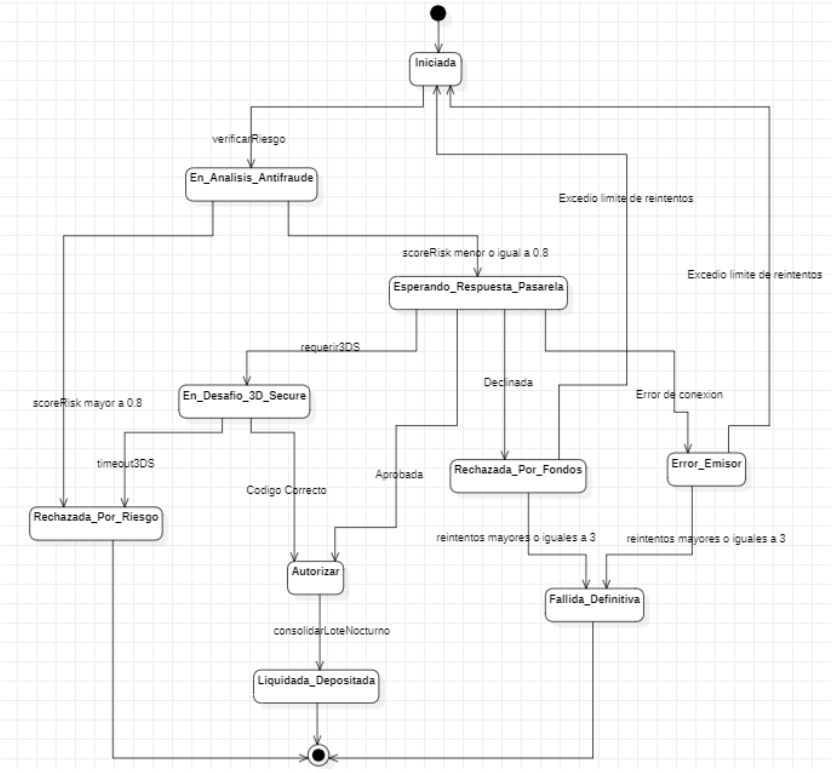
Fichas Técnicas de las 10 Entidades:

Nombre de cada Entidad, Objeto Representado y Diagrama de estados.

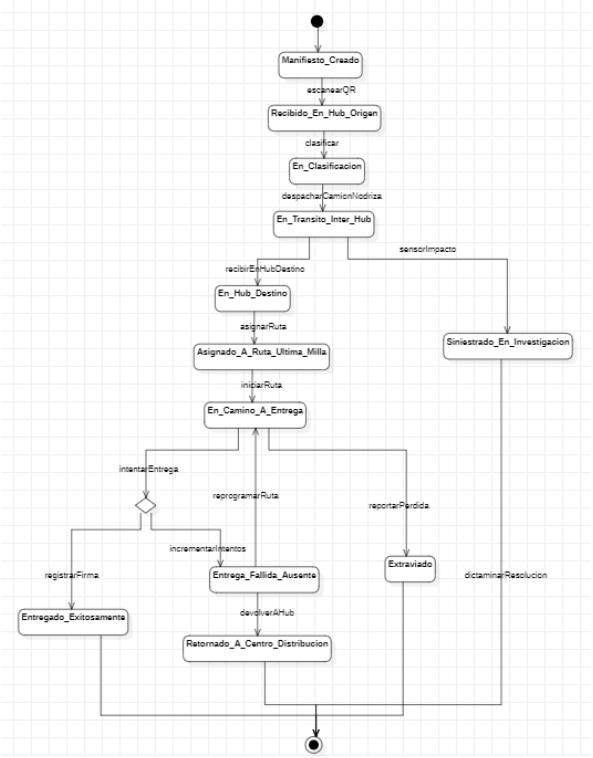
01. OrdenDeCompra – Ciclo transaccional y comercial de la orden.



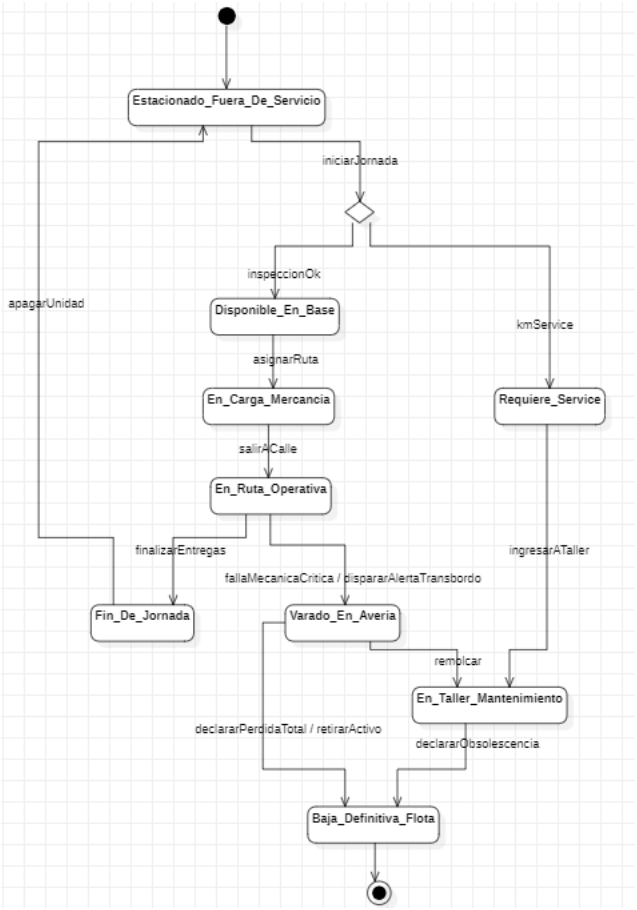
02. TransaccionDePago --> Pasarela bancaria, fraudes, 3DS y liquidación.



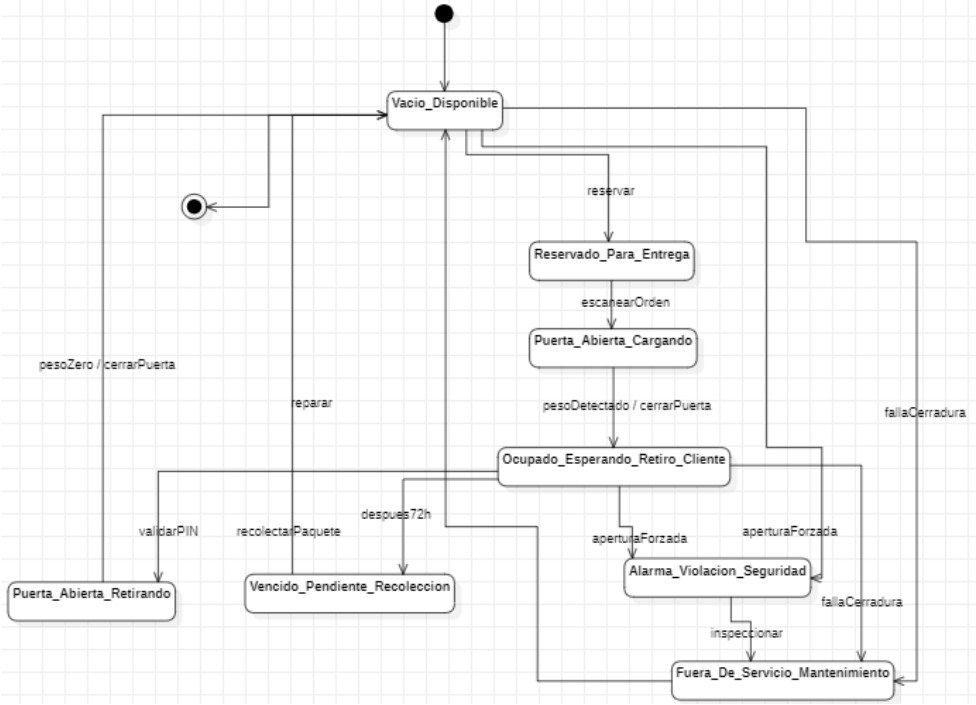
03. PaqueteFisico --> Cadena de custodia logística desde hub a destino.



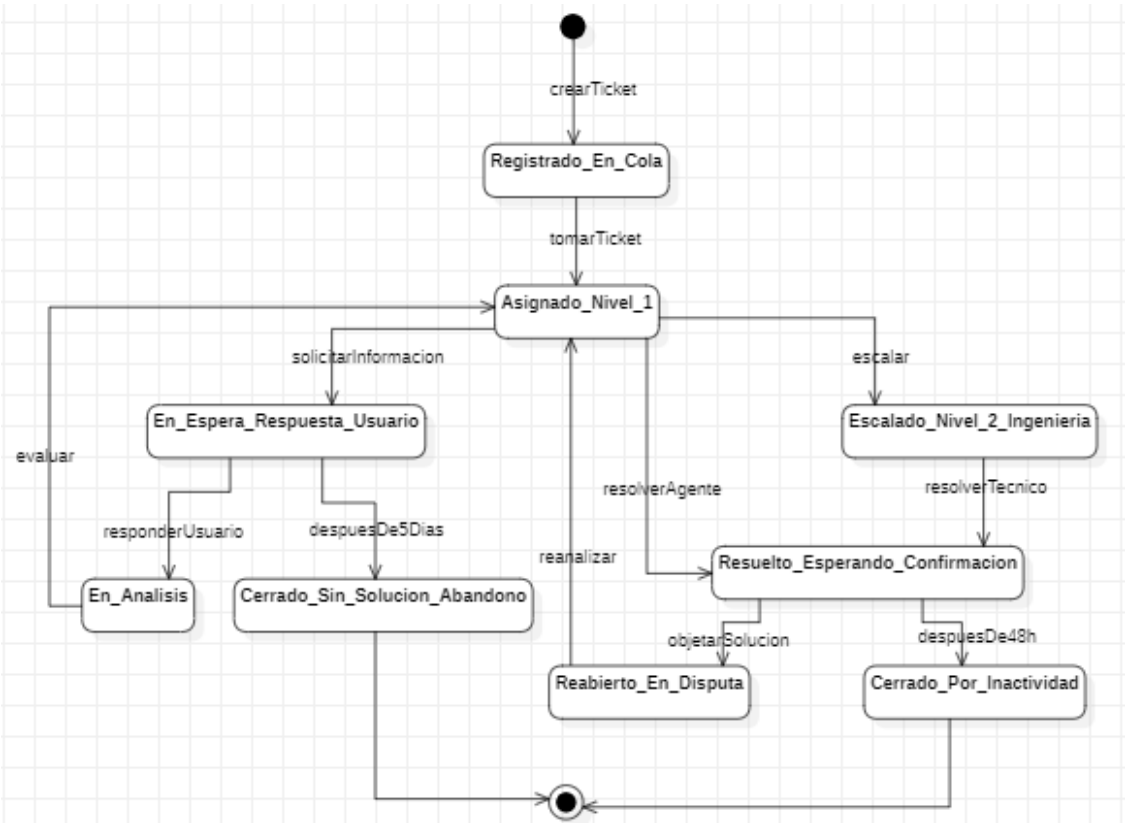
04. VehiculoDeReparto --> Telemetría, mantenimientos y turnos de la unidad.



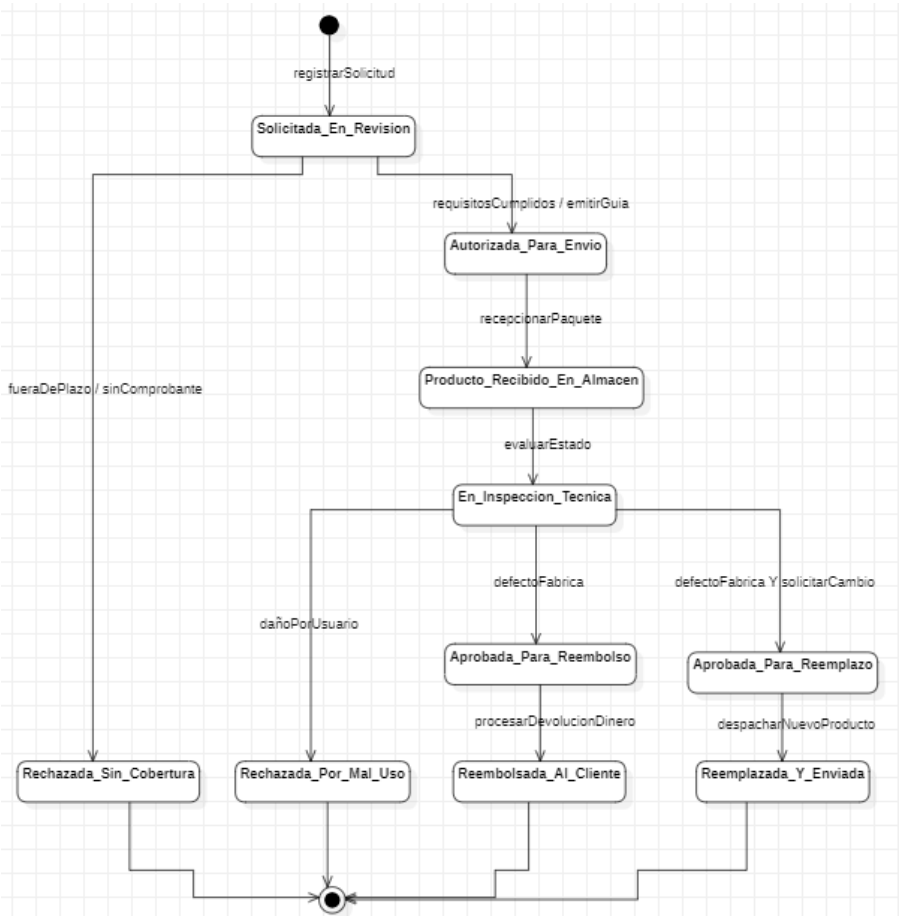
05. CasilleroInteligente (IoT) --> Compartimiento físico, códigos OTP y hardware.



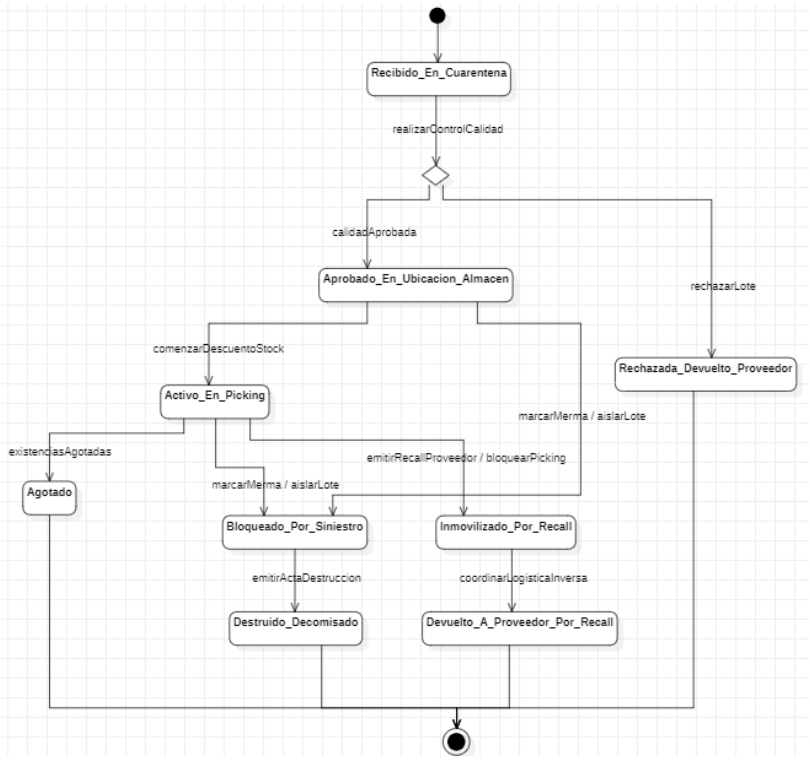
06. TicketDeSoporte --> SLAs de atención al cliente y escalamientos N1/N2.



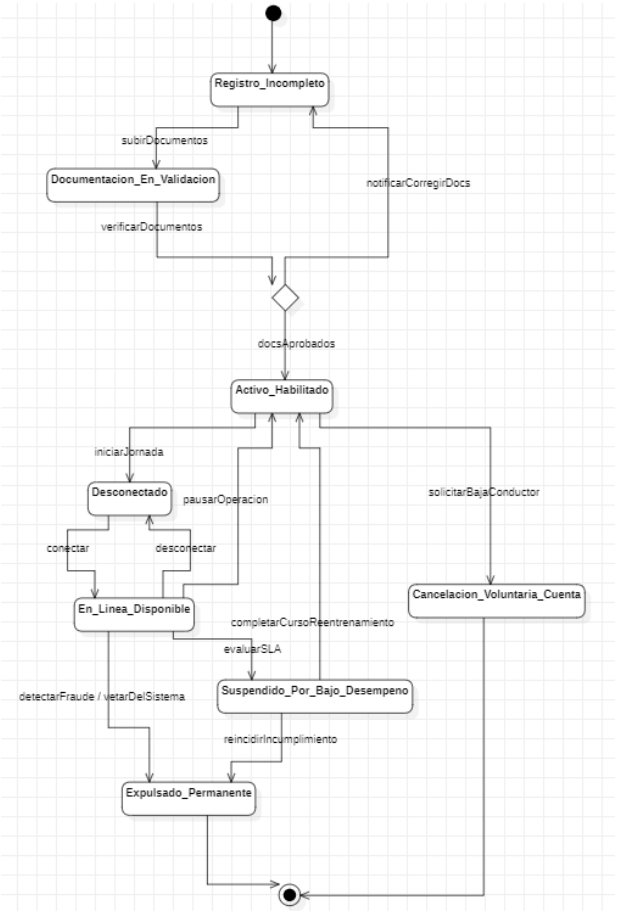
07. SolicitudDeDevolucion --> Logística inversa, inspección y reembolso.



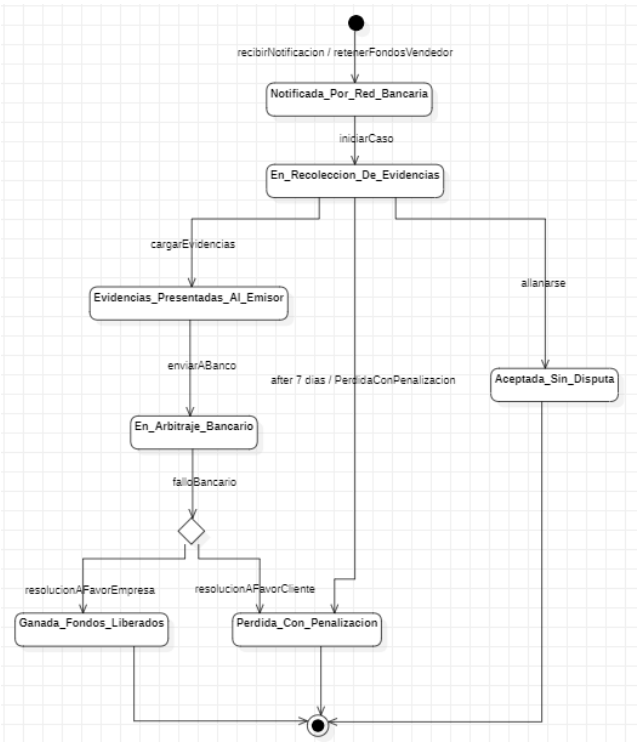
08. LoteDeInventario --> Cuarentena, stock activo, mermas y bloqueo.



09. CuentaDeConductor --> Onboarding, antecedentes, suspensión y baja.



10. DisputaBancaria --> Reclamos por contracargo, evidencias y arbitraje.



Matriz de Transición de Estados y Análisis de Casos de Borde y Resiliencia:

01. OrdenDeCompra

Objeto representado: Entidad agregadora de la compra del cliente; representa el ciclo comercial desde el carrito hasta la entrega o cancelación.

Matriz de transición de estados

Estado origen	Evento disparador	Guarda [Boolean]	Efecto / Acción	Estado destino	Justificación arquitectónica / regla
Borrador_En_Carrito	checkout()	[carritoValido && itemsDisponibles]	crearOrden()	Esperando_Autorizacion_Pago	Solo se inicia el checkout si la orden tiene datos válidos y los artículos pueden procesarse.
Esperando_Autorizacion_Pago	pagoConfirmado()	[respuestaPago == 'APPROVED']	confirmarPago()	En_Procesamiento_Almacen	Evita procesar una orden sin confirmación verificable del pago.
Esperando_Autorizacion_Pago	after(30 min)	[pagoConfirmado == false]	cancelarOrden(); liberarInventario()	Cancelada_Por_Timeout	El vencimiento impide mantener órdenes pendientes indefinidamente y devuelve el inventario.
Despachada_En_Ruta	confirmarEntrega()	[evidenciaEntregaValida]	cerrarOrden()	Completada_Entregada	La orden concluye con evidencia de entrega.

Análisis de casos de borde y resiliencia

En esta entidad, uno de los problemas que se deben prevenir es que la conexión falle justo cuando se está confirmando el pago. En ese caso, no conviene asumir que el pago fue rechazado ni que se aprobó; lo correcto es consultar nuevamente a la pasarela y verificar el resultado. También es importante que el sistema no cree dos órdenes ni procese dos reembolsos cuando el usuario presione

02. TransaccionDePago

Objeto representado: Registro financiero individual que coordina la autorización, autenticación antifraude y liquidación con una pasarela bancaria.

Matriz de transición de estados

Estado origen	Evento disparador	Guarda [Boolean]	Efecto / Acción	Estado destino	Justificación arquitectónica / regla
Iniciada	iniciarAnalisis()	[datosTarjetaValidos]	calcularScoring()	En_Analisis_Antifraude	El análisis de riesgo precede a cualquier autorización.
En_Analisis_Antifraude	resultadoScoring()	[scoringRiesgo == 'BAJO']	solicitarAutorizacion()	Esperando_Respuesta_Pasarela	Solo un riesgo aceptable continúa al proveedor de pagos.
En_Analisis_Antifraude	resultadoScoring()	[scoringRiesgo == 'ALTO']	registrarRechazoRiesgo()	Rechazada_Por_Riesgo	Bloquea operaciones que superan el umbral antifraude.
Esperando_Respuesta_Pasarela	solicitarAutenticacion3DS()	[requiere3DS == true]	enviarDesafio3DS()	En_Desafio_3D_Secure	Exige autenticación reforzada cuando la pasarela lo requiere.

En_Desafio_3D_Secure	validarCodigoSMS()	[codigoValido && desafioVigente]	confirmarAutenticacion()	Autorizada	No se autoriza con códigos inválidos o vencidos.
Esperando_Respuesta_Pasarela	respuestaPasarela()	[respuesta == 'DECLINED' && motivo == 'FUNDS']	registrarDeclinacion()	Rechazada_Por_Fondos	Distingue falta de fondos de otros errores.

Análisis de casos de borde y resiliencia

En los pagos, una caída de internet no significa necesariamente que la operación haya fallado. La pasarela podría haber recibido la solicitud y haber procesado el cobro aunque la respuesta no llegue al sistema. Por eso, antes de intentar cobrar otra vez, se debe consultar el estado de la transacción y utilizar un identificador único para evitar cobros duplicados.

03. PaqueteFisico

Objeto representado: Bulto físico etiquetado con código de barras o QR, con trazabilidad de custodia entre centros y hasta el destinatario.

Matriz de transición de estados

Estado origen	**Evento disparador**	**Guarda [Boolean]**	**Efecto / Acción**	**Estado destino**	**Justificación arquitectónica / regla**
Manifiesto_Creado	escanearEnHubOrigen()	[etiquetaValida && manifiestoExiste]	registrarRecepcion()	Recibido_En_Hub_Origen	La custodia comienza con un escaneo verificable.

Recibido_En_Hub_Origen	iniciarClasificacion()	[destinoIdentificado]	asignarBandaClasificacion()	En_Clasificacion	Evita clasificar paquetes sin destino.
En_Clasificacion	cargarCamionNodriza()	[vehiculoAsignado && manifiestoCerrado]	registrarSalidaHub()	En_Transito_Inter_Hub	Mantiene relación entre paquete, vehículo y manifiesto.
En_Transito_Inter_Hub	escanearEnHubDestino()	[escaneoValido]	registrarLlegadaHub()	En_Hub_Destino	Confirma recepción en el destino antes de la última milla.
En_Hub_Destino	asignarRuta()	[repartidorDisponible && rutaValida]	asociarRuta()	Asignado_A_Ruta_Ultima_Milla	Solo asigna una ruta operativa válida.

Análisis de casos de borde y resiliencia

Si el sistema pierde conexión durante el traslado, los escaneos realizados por el personal o los dispositivos deben guardarse temporalmente y enviarse cuando vuelva la red. Cada registro debe incluir una fecha, una ubicación y un identificador que permita reconocer si el mismo evento ya se recibió. La falta de un escaneo no demuestra por sí sola que el paquete se perdió, por lo que primero debe revisarse la trazabilidad y abrirse una incidencia si hace falta.

04. VehiculoDeReparto

Objeto representado: Activo de la flota que controla disponibilidad, asignación de rutas, operación, averías y mantenimiento.

Matriz de transición de estados

Estado origen	Evento disparador	Guarda [Boolean]	Efecto / Acción	Estado destino	Justificación arquitectónica / regla
Estacionado_Fuera_De_Servicio	iniciarJornada()	[inspeccionTecnicaAprobada && mantenimientoVigente]	habilitarUnidad()	Disponible_En_Base	La unidad solo opera si supera inspección y mantenimiento.
Estacionado_Fuera_De_Servicio	iniciarJornada()	[mantenimientoVigente == false]	bloquearAsignacion()	Requiere_Service	Previene el despacho de vehículos con mantenimiento vencido.
Disponible_En_Base	asignarRuta()	[conductorHabilitado && capacidadSuficiente]	reservarUnidad(); prepararCarga()	En_Carga_Mercancia	Evita asignaciones incompatibles con conductor o capacidad.

Análisis de casos de borde y resiliencia

La seguridad del conductor y de los paquetes debe tener prioridad cuando un vehículo presenta una falla. Si se pierde la señal de GPS o de telemetría, el sistema no debe concluir automáticamente que el vehículo está averiado; debe marcar la información como desactualizada y

solicitar una comprobación. Mientras no se confirme que la unidad está en condiciones, es mejor evitar asignarle nuevas rutas. Si se confirma una avería crítica durante el recorrido, se genera una alerta y se coordina el traslado de los paquetes a otro vehículo.

05. CasilleroInteligente (IoT)

Objeto representado: Compartimiento electromecánico que reserva espacio, recibe y entrega paquetes mediante PIN/QR temporal y sensores.

Matriz de transición de estados

Estado origen	Evento disparador	Guarda [Boolean]	Efecto / Acción	Estado destino	Justificación arquitectónica / regla
Vacio_Disponible	reservarCompartimiento()	[hardwareDisponible && compartimientoLibre]	asignarReserva()	Reservado_Para_Entrega	No reserva un compartimiento ocupado o fuera de servicio.
Reservado_Para_Entrega	abrirParaCarga()	[ordenValida && repartidorAutorizado]	desbloquearPuerta()	Puerta_Abierta_Cargando	La apertura requiere autorización digital.
Puerta_Abierta_Cargando	cerrarPuerta()	[sensorPeso > 0 && cerraduraConfirmada]	registrarIngresoPaquete()	Ocupado_Esperando_Retiro_Cliente	Confirma presencia física y cierre seguro antes de declarar ocupado.

Análisis de casos de borde y resiliencia

El casillero necesita comprobar su estado físico antes de permitir una nueva operación. Por ejemplo, si se va la energía mientras la puerta está abierta, al regresar la electricidad no se debe dar por hecho que el paquete quedó guardado o que el compartimiento está vacío. El sistema debe

revisar los sensores de puerta, peso y cerradura, y comparar esa información con el último registro guardado. Si no puede confirmar que todo está seguro, debe mantener el compartimiento fuera de servicio y avisar al personal responsable. Los mensajes de los sensores también pueden llegar repetidos o con retraso, así que deben identificarse para no ejecutar dos veces la misma acción.

06. TicketDeSoporte

Objeto representado: Incidencia abierta por un usuario para registrar consultas, reclamos o problemas y controlar su atención mediante SLA.

Matriz de transición de estados

Estado origen	**Evento disparador**	**Guarda [Boolean]**	**Efecto / Acción**	**Estado destino**	**Justificación arquitectónica / regla**
Registrado_En_Cola	asignarAgente()	[agenteN1Disponible]	asignarTicket()	Asignado_Nivel_1	La asignación debe quedar vinculada a un agente responsable.
Asignado_Nivel_1	solicitarInformacion()	[informacionInsuficiente]	notificarUsuario(); iniciarPlazoRespuesta()	En_Espera_Respuesta_Usuario	Pausa el análisis mientras se obtiene información necesaria.
En_Espera_Respuesta_Usuario	recibirRespuestaUsuario()	[respuestaValida]	registrarRespuesta()	En_Analisis	Reanuda el análisis cuando llega la información solicitada.

Análisis de casos de borde y resiliencia

En los tickets de soporte, los plazos deben controlarse con cuidado para no cerrar casos que ya recibieron una respuesta. Si falla el envío de una notificación, el sistema debe guardar el incidente y permitir que se vuelva a enviar sin duplicar el ticket ni reiniciar el plazo de manera incorrecta. Los dos tiempos de la práctica tienen propósitos distintos: las 48 horas se aplican cuando el usuario debe confirmar una solución, mientras que los cinco días hábiles corresponden a la falta de respuesta cuando se le ha pedido información.

07. SolicitudDeDevolucion

Objeto representado: Solicitud de logística inversa y garantía que valida elegibilidad, retorno físico e inspección antes del reembolso.

Matriz de transición de estados

Estado origen	Evento disparador	Guarda [Boolean]	Efecto / Acción	Estado destino	Justificación arquitectónica / regla
Solicitada_Por_Cliente	validarSolicitud()	[reciboValido && diasDesdeEntrega <= 30]	revisarEvidencias()	En_Revision_Elegibilidad	La revisión inicia con comprobantes y dentro del plazo de política.
Solicitada_Por_Cliente	validarSolicitud()	[diasDesdeEntrega > 30]	registrarRechazoPolitica()	Rechazada_Fuera_De_Politica	Aplica el límite de 30 días indicado en el caso.
En_Revision_Elegibilidad	aprobarElegibilidad()	[productoElegible && evidenciaSuficiente]	generarGuiaRetorno()	Aprobada_Generando_Guia_Retorno	La guía se emite solo tras aprobar la elegibilidad.

Aprobada_Generando_Guia_Retorno	recibirEnBodega()	[guiaValida && paqueteIdentificado]	registrarRecepcion()	Recibida_En_Inspeccion_Calidad	Vincula el artículo físico con la solicitud autorizada.

Análisis de casos de borde y resiliencia

Si ocurre un problema de conexión al crear la guía de retorno o al realizar el reembolso, la solicitud no debe avanzar hasta confirmar qué pasó. Es necesario consultar al servicio correspondiente antes de repetir la operación, para no generar dos guías ni devolver el dinero dos veces. El límite de 30 días debe calcularse a partir de la fecha de entrega que consta en el sistema, no desde el momento en que el cliente vuelve a enviar la solicitud. Además, la inspección del producto es una parte importante del proceso: si se encuentra manipulado, dañado de forma no cubierta o sustituido por otro objeto, se debe registrar el hallazgo y rechazar la garantía según la regla descrita. En todos los casos, conviene conservar el motivo de aprobación o rechazo para que la decisión pueda revisarse después.

08. LoteDeInventario

Objeto representado: Conjunto de mercancías de un proveedor, controlado desde la recepción y cuarentena hasta el picking, agotamiento o bloqueo.

Matriz de transición de estados

Estado origen	**Evento disparador**	**Guarda [Boolean]**	**Efecto / Acción**	**Estado destino**	**Justificación arquitectónica / regla**
Recibido_En_Cuarentena	aprobarControlCalidad()	[calidadAprobada && loteTrazable]	registrarAprobacion(); asignarUbicacion()	Aprobado_En_Ubicacion_Almacen	El lote no se utiliza hasta superar control de calidad.

Recibido_En_Cuarentena	rechazarControlCalidad()	[calidadAprobada == false]	bloquearDisponibilidad()	Bloqueado_Por_Siniestro	Evita poner en circulación mercancía no conforme; validar el motivo específico en el modelo final.
Aprobado_En_Ubicacion_Almacen	iniciarPicking()	[existencias > 0 && sinBloqueos]	reservarUnidades()	Activo_En_Picking	Solo se extraen unidades de lotes disponibles y sin bloqueos.

Análisis de casos de borde y resiliencia

Cuando hay una interrupción de la red o del sistema de inventario, no se deben descontar unidades si el cambio no quedó guardado correctamente. De lo contrario, dos pedidos podrían reservar el mismo producto o las existencias podrían aparecer en negativo. Por eso, las operaciones de picking deben comprobar que todavía haya unidades disponibles y que el lote no tenga bloqueos. Si se reporta un daño en el almacén o llega una notificación válida de retiro del producto por parte del proveedor, el lote debe dejar de asignarse a nuevos pedidos. La destrucción de un lote bloqueado debe contar con autorización y evidencia del incidente.

09. CuentaDeConductor

Objeto representado: Perfil contractual y operativo del repartidor, desde el registro y validación documental hasta la habilitación, suspensión o expulsión.

Matriz de transición de estados

Estado origen	Evento disparador	Guarda [Boolean]	Efecto / Acción	Estado destino	Justificación arquitectónica / regla
Registro_Incompleto	subirDocumentos()	[DNI && licencia && antecedentesPresentes]	enviarACompliance()	Documentacion_En_Validacion	La validación inicia cuando se presenta el conjunto documental requerido.
Documentacion_En_Validacion	aprobarCompliance()	[documentosValidos && antecedentesAprobados]	habilitarCuenta()	Activo_Habilitado	Solo documentos verificados habilitan al conductor.
Activo_Habilitado	conectarse()	[cuentaVigente && sinSuspension]	marcarDisponible()	En_Linea_Disponible	La conexión requiere una cuenta operativa y sin sanciones.

Análisis de casos de borde y resiliencia

Si el servicio que revisa los documentos está fuera de línea, la cuenta del conductor no debe activarse automáticamente. Es mejor mantenerla pendiente hasta confirmar que el DUI, la licencia y los antecedentes cumplen con los requisitos. La calificación y las quejas también deben

calcularse con información confiable, ya que pueden provocar una suspensión. Antes de desconectar a un conductor, se debe comprobar que no tenga una entrega pendiente o reasignar la ruta para que los paquetes no queden sin responsable.

10. DisputaBancaria

Objeto representado: Proceso de contracargo iniciado por el banco emisor, con retención de fondos, recolección de evidencias y decisión de arbitraje.

Matriz de transición de estados

Estado origen	Evento disparador	Guarda [Boolean]	Efecto / Acción	Estado destino	Justificación arquitectónica / regla
Notificada_Por_Red_Bancaria	abrirExpediente()	[notificacionAutentica && disputaNoDuplicada]	retenerFondos(); crearExpediente()	En_Recoleccion_De_Evidencias	Se valida el aviso y se retienen fondos mientras se investiga.
En_Recoleccion_De_Evidencias	presentarEvidencias()	[evidenciasCompletas && dentroDePlazo]	enviarAlEmisor(); registrarAcuse()	Evidencias_Presentadas_Al_Emisor	Las evidencias deben presentarse completas y dentro del plazo.
En_Recoleccion_De_Evidencias	after(7 dias)	[evidenciasNoEnviadas]	registrarIncumplimiento(); aplicarPenalizacion()	Perdida_Con_Penalizacion	Evita que el expediente pierda el plazo por inacción.

Evidencias_Presentadas_Al_Emisor	iniciarArbitraje()	[emisorAceptaExpediente]	registrarInicioArbitraje()	En_Arbitraje_Bancario	Formaliza la etapa de evaluación bancaria.
En_Recoleccion_De_Evidencias	allanarse()	[costoLitigio > montoEnDisputa && decisionAutorizada]	aceptarContracargo()	Aceptada_Sin_Disputa	Permite aceptar el reclamo cuando litigar no resulta conveniente.

Análisis de casos de borde y resiliencia

Las disputas bancarias tienen plazos importantes, por lo que el sistema debe registrar cuándo recibe la notificación y evitar crear expedientes duplicados. Si la conexión falla al enviar las evidencias, estas deben conservarse y el envío debe repetirse de forma controlada hasta obtener una confirmación del banco. No basta con que el sistema haya iniciado la solicitud; debe quedar constancia de que el emisor la recibió. También conviene generar avisos antes de que se cumplan los siete días disponibles para presentar la documentación. Si el plazo vence sin que se envíen las evidencias, la disputa pasa a pérdida con penalización.

Diseño de Persistencia y Auditoría de Estados

Objetivo

El objetivo del diseño de persistencia y auditoría de estados es mantener un registro histórico de los cambios que experimentan las entidades del sistema LogiFlow 360°. Esto permite conocer cuándo ocurrió una transición, cuál era el estado anterior, cuál fue el nuevo estado, qué evento provocó el cambio y qué usuario o componente del sistema fue responsable.

La auditoría es importante para detectar errores, investigar incidentes y verificar que las operaciones se hayan ejecutado correctamente. También facilita el seguimiento de situaciones como cancelaciones, cambios en el inventario, fallos de entrega y problemas relacionados con pagos o seguridad.

Script SQL de auditoría

El siguiente script está diseñado para SQL Server. Crea una tabla para almacenar el historial de las transiciones de estado.

```
CREATE TABLE historial_transiciones_estado (
    id_transicion BIGINT IDENTITY(1,1) PRIMARY KEY,
    entidad VARCHAR(100) NOT NULL,
    id_entidad VARCHAR(100) NOT NULL,
    estado_origen VARCHAR(100),
    estado_destino VARCHAR(100) NOT NULL,
    evento VARCHAR(150) NOT NULL,
    fecha_hora DATETIME2 NOT NULL
        DEFAULT SYSDATETIME(),
    responsable VARCHAR(150) NOT NULL,
    payload_evento NVARCHAR(MAX),

    CONSTRAINT CK_payload_json
    CHECK (
        payload_evento IS NULL
        OR ISJSON(payload_evento) = 1
    )
);
```

El siguiente ejemplo muestra cómo registrar una transición de una orden de compra después de confirmar un pago.

```sql
INSERT INTO historial_transiciones_estado (
    entidad,
    id_entidad,
    estado_origen,
    estado_destino,
    evento,
    responsable,
    payload_evento
)
VALUES (
    'OrdenDeCompra',
    'ORD-1001',
    'Pendiente_De_Pago',
    'Aprobada_Para_Despacho',
    'confirmarPago()',
    'ServicioPagos',
    N'{"resultado":"aprobado","referencia":"PAY-1001"}'
);
```

Para consultar el historial de una entidad específica se puede utilizar la siguiente consulta:

```sql
SELECT
    id_transicion,
    entidad,
    id_entidad,
    estado_origen,
    estado_destino,
    evento,
    fecha_hora,
    responsable,
    payload_evento
FROM historial_transiciones_estado
WHERE entidad = 'OrdenDeCompra'
  AND id_entidad = 'ORD-1001'
ORDER BY fecha_hora ASC, id_transicion ASC;
```

Relación con las máquinas de estados

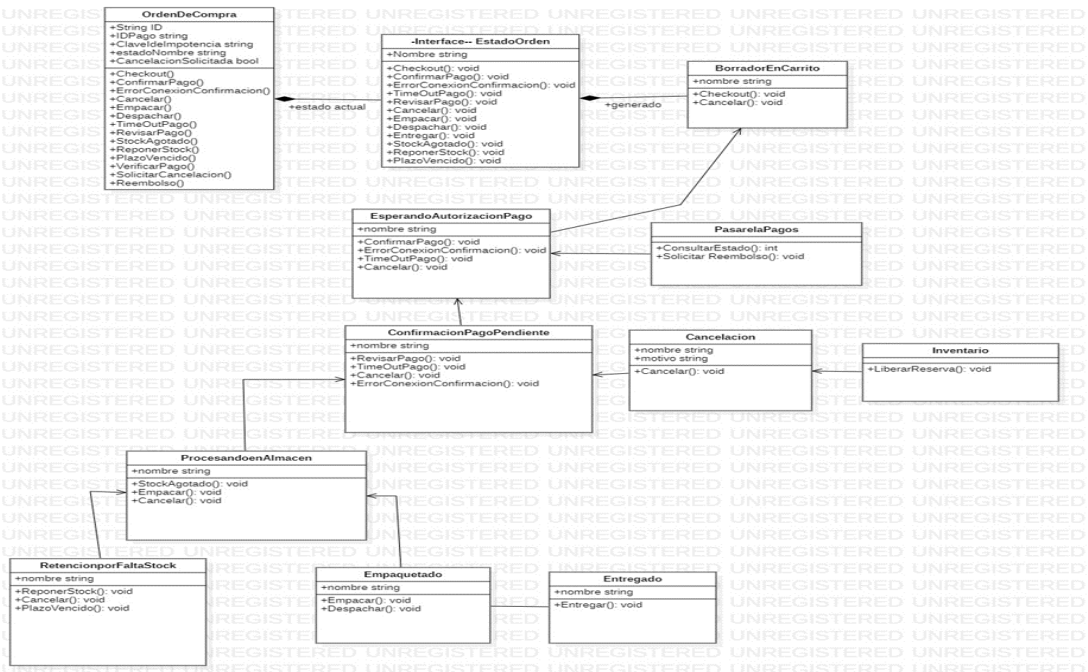
La tabla permite relacionar la documentación UML con los cambios registrados durante la ejecución del sistema. Cada vez que una transición válida se complete, el componente responsable deberá registrar el estado anterior, el nuevo estado y el evento correspondiente.

Es importante aclarar que la creación de la tabla no registra los cambios automáticamente. Para que la auditoría funcione, el backend deberá insertar los registros durante la ejecución de las transiciones o mediante un mecanismo de auditoría debidamente configurado.

Propuesta de Implementación de Código (Patrón State):

Se implementó el patrón de diseño State sobre la entidad OrdenDeCompra de la plataforma LogiFlow 360°. La clase OrdenDeCompra funciona como contexto y delega las operaciones al objeto que representa su estado actual. Cuando se intenta ejecutar una operación no admitida, se genera una TransicionInvalidaException, evitando que la orden avance a un estado inconsistente. El proceso continua desde que el borrador en carrito se convierte en una orden, pase por todos los procesos necesarios para ser entregada al cliente o cancelada si es necesario.

Diagrama de Clase - OrdenDeCompra



A continuación, explicaremos el código en el cual se ejecutará el diagrama incluyendo todo lo necesario. El código se desarrolló bajo el formato de C#

OrdenDeCompra.cs: Es la orden real. No hace la lógica, solo guarda en qué estado está el pedido (EstadoActual) y tiene el la posibilidad de cambiarlo bajo el método CambiarEstado().

```
namespace Orden_de_Compra_3_ev
{
    99+ referencias
    public class OrdenDeCompra
    {
        17 referencias
        public EstadoOrden EstadoActual { get; private set; }

        1 referencia
        public OrdenDeCompra()
        {
            EstadoActual = new BorradorEnCarrito();
        }

        11 referencias
        public void CambiarEstado(EstadoOrden nuevo)
        {
            Console.WriteLine($"Cambio de estado a: {EstadoActual.Nombre} -> {nuevo.Nombre}");
            EstadoActual = nuevo;
        }

        1 referencia
        public void Checkout() => EstadoActual.Checkout(this);
        10 referencias
        public abstract void Despachar(OrdenDeCompra orden);
        10 referencias
        public abstract void Entregar(OrdenDeCompra orden);
        9 referencias
        public abstract void StockAgotado(OrdenDeCompra orden);
        9 referencias
        public abstract void ReponerStock(OrdenDeCompra orden);
        9 referencias
        public abstract void PlazoVencido(OrdenDeCompra orden);
        }
    }
    1 referencia
    public void Despachar() => EstadoActual.Despachar(this);
    1 referencia
    public void Entregar() => EstadoActual.Entregar(this);
}
```

EstadoOrden.cs: Es la plantilla. Acá estarán todos los métodos que se pueden hacer en la orden. Desde Checkout(), ConfirmarPago(), Empacar() hasta Despachar(), Entregar(), Cancelar(), etc

```
namespace Orden_de_Compra_3_ev
{
    2 referencias
    public class BorradorEnCarrito : EstadoOrden
    {
        18 referencias
        public override string Nombre { get { return "BorradorEnCarrito"; } }

        2 referencias
        public override void Checkout(OrdenDeCompra o)
        {
            Console.WriteLine("Checkout OK");
            o.CambiarEstado(new EsperandoAutorizacionPago());
        }

        2 referencias
        public override void Cancelar(OrdenDeCompra o)
        {
            o.CambiarEstado(new Cancelacion("Carrito abandonado"));
        }
        2 referencias
        2 referencias
        public override void ConfirmarPago(OrdenDeCompra o) { throw new TransicionInvalidaException("No se puede ConfirmarPago desde " + Nombre); }
        2 referencias
        public override void ErrorConexionConfirmacion(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
        2 referencias
        public override void TimeOutPago(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
        2 referencias
        public override void RevisarPago(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
        2 referencias
        public override void Empacar(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
        2 referencias
        public override void Despachar(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
        2 referencias
        public override void Entregar(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
        1 referencia
        public override void StockAgotado(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
        1 referencia
        public override void ReponerStock(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
        1 referencia
        public override void PlazoVencido(OrdenDeCompra o) { throw new TransicionInvalidaException("No permitido desde " + Nombre); }
    }
}
```

BorradorEnCarrito: Como se indica en el trabajo es donde inicia todo. Solo puede hacer Checkout() para pasar a EsperandoAutorizacionPago, y Cancelar().

EsperandoAutorizacionPago: En esta parte del proceso ya se pasó el checkout. Acá se espera que se reciba el dinero y que el banco confirme. Puede: ConfirmarPago para ir a Revision, o si no se paga a tiempo en el método TimeOutPago()se cancela, ErrorConexion es para volver a intentar, y Cancelar() para abortar el intento.



```
namespace Orden_de_Compra_3_ev
{
    1 referencia
    public class EsperandoAutorizacionPago : EstadoOrden
    {
        8 referencias
        public override string Nombre => "EsperandoAutorizacionPago";
        2 referencias
        public override void Checkout(OrdenDeCompra o) { throw new System.Exception("No"); }
        2 referencias
        public override void ConfirmarPago(OrdenDeCompra o) { }
        2 referencias
        public override void ErrorConexionConfirmacion(OrdenDeCompra o) { }
        2 referencias
        public override void TimeOutPago(OrdenDeCompra o) { }
        2 referencias
        public override void RevisarPago(OrdenDeCompra o) { }
        2 referencias
        public override void Cancelar(OrdenDeCompra o) { }
        2 referencias
        public override void Empacar(OrdenDeCompra o) { }
        2 referencias
        public override void Despachar(OrdenDeCompra o) { }
        2 referencias
        public override void Entregar(OrdenDeCompra o) { }
        1 referencia
        public override void StockAgotado(OrdenDeCompra o) { }
        1 referencia
        public override void ReponerStock(OrdenDeCompra o) { }
        1 referencia
        public override void PlazoVencido(OrdenDeCompra o) { }
    }
}
```

PagoenRevision.cs : Acá es donde ya se cuenta con el dinero y ahora un humano lo revisa Se revisa el pago y si no hay problema pasa a Pago Aceptado



```
namespace Orden_de_Compra_3_ev
{
    0 referencias
    public class PagoEnRevision : EstadoOrden
    {
        8 referencias
        public override string Nombre => "PagoEnRevision";
        2 referencias
        public override void RevisarPago(OrdenDeCompra o) { Console.WriteLine("Pago aceptado"); o.CambiarEstado(new PagoAceptado()); }
        2 referencias
        public override void Cancelar(OrdenDeCompra o) { o.CambiarEstado(new Cancelacion("Rechazo revision")); }
        2 referencias
        public override void Checkout(OrdenDeCompra o) => throw new TransicionInvalidaException("Ya pagado");
        2 referencias
        public override void ConfirmarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("Ya confirmado");
        2 referencias
        public override void ErrorConexionConfirmacion(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void TimeOutPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void Empacar(OrdenDeCompra o) => throw new TransicionInvalidaException("Falta aprobar");
        2 referencias
        public override void Despachar(OrdenDeCompra o) => throw new TransicionInvalidaException("Falta aprobar");
        2 referencias
        public override void Entregar(OrdenDeCompra o) => throw new TransicionInvalidaException("Falta aprobar");
        1 referencia
        public override void StockAgotado(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void ReponerStock(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void PlazoVencido(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
    }
}
```

PagoAceptado: En esta parte del proceso ya no hay ningún problema con el pago. Ya se puede Empacar y ya no se puede cancelar.

```
namespace Orden_de_Compra_3_ev
{
    1 referencia
    public class PagoAceptado : EstadoOrden
    {
        8 referencias
        public override string Nombre => "PagoAceptado";
        2 referencias
        public override void Empacar(OrdenDeCompra o) { o.CambiarEstado(new EnEmpaque()); }
        1 referencia
        public override void StockAgotado(OrdenDeCompra o) { o.CambiarEstado(new ErrorTransaccion("Sin stock")); }
        2 referencias
        public override void Cancelar(OrdenDeCompra o) { o.CambiarEstado(new Cancelacion("Anulado post pago")); }
        2 referencias
        public override void Checkout(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void ConfirmarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void ErrorConexionConfirmacion(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void TimeOutPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void RevisarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void Despachar(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void Entregar(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void ReponerStock(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void PlazoVencido(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
    }
}
```

EnEmpaque: Cuando el producto se encuentra en bodega y pasa a ser despachado

```
namespace Orden_de_Compra_3_ev
{
    1 referencia
    public class EnEmpaque : EstadoOrden
    {
        8 referencias
        public override string Nombre => "EnEmpaque";
        2 referencias
        public override void Despachar(OrdenDeCompra o) { o.CambiarEstado(new Enviado()); }
        2 referencias
        public override void Checkout(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void ConfirmarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void ErrorConexionConfirmacion(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void TimeOutPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void RevisarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void Cancelar(OrdenDeCompra o) => throw new TransicionInvalidaException("No se puede cancelar en empaque");
        2 referencias
        public override void Empacar(OrdenDeCompra o) => throw new TransicionInvalidaException("Ya en empaque");
        2 referencias
        public override void Entregar(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void StockAgotado(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void ReponerStock(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void PlazoVencido(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
    }
}
```

Enviado.cs: Sale de bodega y va en camino con el courier. Solo puede Entregarse

```
namespace Orden_de_Compra_3_ev
{
    1 referencia
    public class Enviado : EstadoOrden
    {
        8 referencias
        public override string Nombre => "Enviado";
        2 referencias
        public override void Entregar(OrdenDeCompra o) { o.CambiarEstado(new Entregada()); }
        2 referencias
        public override void Checkout(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void ConfirmarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void ErrorConexionConfirmacion(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void TimeOutPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void RevisarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void Cancelar(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void Empacar(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        2 referencias
        public override void Despachar(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void StockAgotado(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void ReponerStock(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
        1 referencia
        public override void PlazoVencido(OrdenDeCompra o) => throw new TransicionInvalidaException("No valido");
    }
}
```

Entregada: Estado final. Ya no hace nada más. Acá termina el proceso y el producto ya está con el cliente

```
namespace Orden_de_Compra_3_ev
{
    1 referencia
    public class Entregada : EstadoOrden
    {
        8 referencias
        public override string Nombre => "Entregada - FINAL";
        2 referencias
        public override void Checkout(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        2 referencias
        public override void ConfirmarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        2 referencias
        public override void ErrorConexionConfirmacion(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        2 referencias
        public override void TimeOutPago(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        2 referencias
        public override void RevisarPago(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        2 referencias
        public override void Cancelar(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        2 referencias
        public override void Empacar(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        2 referencias
        public override void Despachar(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        2 referencias
        public override void Entregar(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        1 referencia
        public override void StockAgotado(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        1 referencia
        public override void ReponerStock(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
        1 referencia
        public override void PlazoVencido(OrdenDeCompra o) => throw new TransicionInvalidaException("Finalizado");
    }
}
```

Clase Principal: Acá es donde se mandan a llamar todos los métodos y se ejecuta el programa

```
namespace Orden_de_Compra_3_ev
{
    0 referencias
    internal class Program
    {
        0 referencias
        static void Main()
        {
            var orden = new OrdenDeCompra();
            Console.WriteLine($"Inicial: {orden.EstadoActual.Nombre}");
            orden.Checkout();
            Console.WriteLine($"Despues Checkout: {orden.EstadoActual.Nombre}");
            orden.ConfirmarPago();
            Console.WriteLine($"Despues Confirmar: {orden.EstadoActual.Nombre}");
            orden.RevisarPago();
            Console.WriteLine($"Despues Revisar: {orden.EstadoActual.Nombre}");
            orden.Empacar();
            orden.Despachar();
            orden.Entregar();
            Console.WriteLine($"FINAL: {orden.EstadoActual.Nombre}");
        }
    }
}
```

Resultado de la ejecución

```
Consola de depuración de Microsoft Visual Studio
Inicial: BorradorEnCarrito
Checkout OK
Cambio de estado a: BorradorEnCarrito -> EsperandoAutorizacionPago
Despues Checkout: EsperandoAutorizacionPago
Despues Confirmar: EsperandoAutorizacionPago
Despues Revisar: EsperandoAutorizacionPago
FINAL: EsperandoAutorizacionPago

C:\Users\HP\OneDrive\Escritorio\Ciclo 02-2026\tercera evaluacion uml\Orden de Compra 3 ev\Orden de Compra 3 ev\bin\Debug
\Orden de Compra 3 ev.exe (proceso 1448) se cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

Lecciones Aprendidas y Conclusiones del Análisis

Lecciones aprendidas:

Durante el desarrollo de las máquinas de estados UML de LogiFlow 360° aprendimos cómo representar las diferentes etapas por las que puede pasar una entidad dentro de un sistema. Por ejemplo, una orden de compra puede estar pendiente de pago, ser aprobada, cancelarse o quedar retenida por falta de inventario, dependiendo de lo que ocurra durante el proceso.

Una de las cosas más importantes que aprendimos fue la diferencia entre los flujos funcionales y los ciclos de vida de los objetos. Los flujos funcionales muestran los pasos o actividades que se realizan para completar un proceso, mientras que las máquinas de estados muestran las diferentes situaciones en las que puede encontrarse una entidad y cómo cambia de una a otra. Aunque ambos tipos de diagramas son útiles, cada uno tiene un propósito diferente.

Conclusiones del análisis:

Con el análisis de las máquinas de estados de LogiFlow 360° logramos comprender mejor cómo se puede representar el comportamiento de las distintas entidades del sistema. Estos diagramas ayudan a identificar los estados por los que puede pasar cada entidad y las diferentes situaciones que pueden presentarse durante su funcionamiento.

Concluimos que los diagramas de actividades y las máquinas de estados no son lo mismo. Los primeros se enfocan en los pasos que se siguen para realizar una actividad, mientras que las máquinas de estados se centran en cómo cambia una entidad según los eventos que ocurren. Por eso, es importante saber cuándo utilizar cada tipo de diagrama para representar correctamente el sistema.